

Comandos básicos: E/S, Condição e Repetição

Primeiros passos em programação

- Vimos até agora como computar algumas expressões simples.
 - Expressões são escritas e computadas imediatamente.
 - Variáveis podem ser usadas para guardar valores temporários.
- Além de expressões simples e variáveis, um programa típico usa vários tipos de construções, como:
 - Comandos condicionais.
 - Comandos de repetição.
 - Estruturas de dados.
 - Definição e uso de funções.
 - Definição e uso de classes e objetos (programação OO).

Primeiros passos em programação

```

>>> # Série de Fibonacci
... a,b=0,1
>>> while b < 10:
...     print(b)
...     a,b=b,a+b

```

The diagram illustrates the following concepts in the code:

- Comentário**: Points to the line `# Série de Fibonacci`.
- Atribuição dupla**: Points to the line `a,b=0,1`.
- Comando de repetição (while)**: Points to the line `while b < 10:`.
- Expressão booleana ou Condição lógica**: Points to the condition `b < 10`.
- Bloco de comandos**: Points to the indented block `print(b)` and `a,b=b,a+b`.
- Identação**: Points to the indentation of the commands inside the while loop.
- Resultado**: Points to the output sequence: `1`, `1`, `2`, `3`, `5`, `8`.

Programas armazenados

- À medida que os programas vão se tornando mais complicados, é mais interessante guardá-los em arquivos e executá-los quando necessário.
- Para escrever e executar nossos códigos usaremos o IDE PyCharm ou VS Code.
- Vamos criar e salvar o arquivo **fibo.py**

```

01. # Série de Fibonacci (< 10)(valores < 1)
02. a, b = 0, 1
03. while b < 10:
04.     print(b)
05.     a, b = b, a+b

```

Comando `print()`

- A função **`print()`** exibe o conteúdo/resultado das expressões na tela seguidos de uma quebra de linha.

- Formas gerais no Python 3:

```
print(argumentos)
```

```
print("string de formato".format(argumentos))
```

```
print(f"string {argumentos}")
```

- Os valores das expressões são escritos um após o outro, na ordem dos *argumentos*.

Comando `print()`

- Exemplos:

```
>>> print(3**2)
9
```

```
>>> print(str(3)+" ao quadrado é "+str(3**2))
3 ao quadrado é 9
```

```
>>> print("{} ao quadrado é {}".format(3, 3**2))
3 ao quadrado é 9
```

```
>>> print(f"3 ao quadrado é {3**2}")
3 ao quadrado é 9
```

Comando `print()`

- Podemos especificar um terminador de linha que não seja a quebra de linha ("`\n`") através do parâmetro **end**:

```
# Com quebra de linha (duas linhas)
>>> print("Uespi - ")
>>> print("Campus de Parnaíba")
Uespi -
Campus de Parnaíba

# Sem quebra de linhas (mesma linha)
>>> print("Uespi - ", end=" ")
>>> print("Campus de Parnaíba")
Uespi - Campus de Parnaíba
```

Comando `print()`

- Mais exemplos:

```
# Sem quebra de linhas (mesma linha)
>>> print("Uespi", end=", ")
>>> print("Campus de Parnaíba")
Uespi, Campus de Parnaíba

# Sem quebra de linhas (mesma linha)
>>> print("Uespi", end=" - ")
>>> print("Campus de Parnaíba")
Uespi - Campus de Parnaíba
```

Comando `input()`

- Qual o parâmetro de término do programa da série de Fibonacci (programa **`fibo.py`**) ?
- Que tal se o usuário pudesse informar o parâmetro de final da sequência de Fibonacci para término do programa?
- O comando **`input`** permite perguntar ao usuário um valor (normalmente é atribuído a uma variável) **na formato de string**.
 - Formato: ***`variável`*** = **`input(pergunta)`**
 - Onde, ***`pergunta`*** é uma string opcional que será exibida para indicar o valor que se espera;
 - ***`variável`*** vai receber o valor informado pelo usuário.

Comando `input()`

- Exemplo:

```
>>> # Série de Fibonacci:
>>> a, b = 0, 1
>>> n = int(input("Entre com um número: "))
    Entre com um número: 10

>>> while b < n:
...     print(b, end=" ")
...     a, b = b, a+b
...
    1 1 2 3 5 8
```

O usuário digita

- Podemos digitar o código acima em uma das IDE adotadas.

Repetição while

- O comando **while** repete uma sequência de comandos sempre que a **expressão booleana** é avaliada como **True**.

- Formato:

```
while expressao:
    comando_1
    comando_2
    ...
    comando_n
```

- Exemplo:

```
>>> a = 10
>>> while a > 7:
...     print(a)
...     a = a - 1
...
10
9
8
```

Repetição: laços infinitos

- Em todo comando de repetição evite os “laços infinitos”.
- Exemplo:

```
a = 10
while a > 8:
    print(a, end=" ")
    a = a + 1
...
10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 ... e continua ... e continua ...
```

Comando condicional: if

- **Caso 1:** executa **comando_1** e **comando_2** apenas se **expressao** for avaliada como **True**. O **comando_3** e o **comando_4** sempre serão executados.

```
if expressao:
    comando_1
    comando_2
comando_3
comando_4
...
comando_n
```

```
a = int(input("Informe um número: "))
if a < 0:
    print(f"{a} é negativo")
print("Obrigado!")
```

- **Execução 1:**
Informe um número: 2
Obrigado!
- **Execução 2:**
Informe um número: -2
-2 é negativo
Obrigado!

Comando condicional: if

- **Caso 2:** executa **comando_1** apenas se **expressao** for avaliada como **True** caso contrário, executa **comando_2**.
- O **comando_3** sempre será executado.

```
if expressao:
    comando_1
else:
    comando_2
comando_3
...
comando_n
```

```
a = int(input("Entre um número: "))
if a < 0:
    print(f"{a} é negativo")
else:
    print(f"{a} é zero ou positivo")
print "Obrigado!"
```

- **Execução 1:** Entre um número: 2
2 é zero ou positivo
Obrigado!
- **Execução 2:** Entre um número: -2
-2 é negativo
Obrigado!

Comando condicional: if

- **Caso 3:** executa **comando_1** apenas se **expressao1** for avaliada como **True** caso contrário, executa **comando_2** apenas se **expressao2** for avaliada como **True** caso contrário, executa **comando_3**. O **comando_4** sempre será executado.

```
if expressao1:
    comando_1
elif expressao2:
    comando_2
else:
    comando_3
comando_4
```

Comando condicional: if – ex. caso 3

```
a = int(input("Informe um número: "))
if a < 0:
    print(f"{a} é negativo")
elif a > 0:
    print("{a} é positivo")
else:
    print("{a} é zero")
print("Obrigado!")
```

- **Execução 1:** Informe um número: 0
0 é zero
Obrigado!
- **Execução 2:** Informe um número: 2
2 é positivo
Obrigado!

Exercícios

- Fazer um programa que mostra qual o maior de dois números informados pelo usuário.
- Fazer um programa que informa se um número positivo lido é primo ou não.
- Fazer um programa que escreva a representação em algarismos romanos de um número inteiro positivo informado pelo usuário.

Exercício: maior de dois números

- Fazer um programa que mostra qual o maior de dois números lidos.
 - O usuário deve informar os dois números (input)
 - O resultado deve ser impresso no console (print)
- Exemplo de entrada:
Entre com o primeiro número: 9
Entre com o segundo número: 6
- Saída:
O maior valor é 9

Exercício: maior de dois números

- Vejamos o seguinte algoritmo:
 - Se o primeiro número é maior, mostre-o.
 - Caso contrário, mostre o segundo número.
- E se os números forem iguais?

Exercício: números primos

- Fazer um programa que informa se um número positivo dado é primo ou não.
 - Entrada: número inteiro positivo.
 - Saída: diagnóstico de primalidade do número.
- Exemplos de execução:

Entre com um número inteiro positivo: 15
15 não é primo, é múltiplo de 3

Entre com um número inteiro positivo: 11
11 é primo

Exercício: números primos

- Um número natural é primo se é divisível apenas por si mesmo e pela unidade.
- Isto sugere o seguinte algoritmo:
 - Se o número é 1, então não é primo
 - Se o número é 2, então é primo
 - Caso contrário,
 - Seja **div** um possível divisor, cujo valor é inicialmente 2
 - Repetir:
 - Se o resto da divisão do número por **div** é zero, então o número não é primo
 - Caso contrário, incrementar **div**
 - Se **div** é maior que a metade do número, então terminar repetição diagnosticando que o número é primo.

Exercício: algoritmos romanos

- Fazer um programa que escreva a representação em algarismos romanos de um número inteiro positivo
 - O usuário deve entrar com um número (input)
 - O resultado deve ser impresso no console (print)
- Exemplo de entrada:
Entre com um número positivo: 1985
- Saída:
1985 em algarismos romanos é MCMLXXXV

Exercício: algoritmos romanos

- Algoritmo
 - A representação em romanos é uma string à qual é acrescentada uma letra por vez
 - Inicialmente, uma string vazia
 - Examinar as sucessivas potências de 10
 - Por exemplo, a letra 'M' corresponde à casa dos milhares
 - Se o número é 2200, sabemos que teremos dois M's na representação em romanos
 - Sabemos que há M's se o número é maior ou igual a 1000
 - Sempre que um milhar for computado, subtrair 1000 do número
- Um processamento semelhante é feito para outros algoritmos romanos, por exemplo:
 - Se o número é maior ou igual que 500, acrescentar 'D'
 - Se o número é maior que 899 e menor que 1000, acrescentar 'CM'

Exercício: algoritmos romanos

- DICA: processando um número entre 1 e 9

```

if num >= 9:
    romano = romano + "IX"
    num = num - 9
if num >= 5:
    romano = romano + "V"
    num = num - 5
if num >= 4:
    romano = romano + "IV"
    num = num - 4
while num >= 1:
    romano = romano + "I"
    num = num - 1
  
```